An opinionated introduction to Reinforcement Learning

Fernando E. Rosas[1]

Iliad Intensive

September 12, 2026

[1] Based on slides from David Quarel

What is Reinforcement Learning?

A lot of machine learning focuses on supervised (or self-supervised) learning.

1. We have access to labelled training data
2. Agent only sees "real world" once it's already performing well
3. Data is i.i.d between batches (or even samples)
4. No planning required (future predictions don't depend on past predictions)

What is Reinforcement Learning?

A lot of machine learning focuses on supervised (or self-supervised) learning.

1. We have access to labelled training data
2. Agent only sees "real world" once it's already performing well
3. Data is i.i.d between batches (or even samples)
4. No planning required (future predictions don't depend on past predictions)

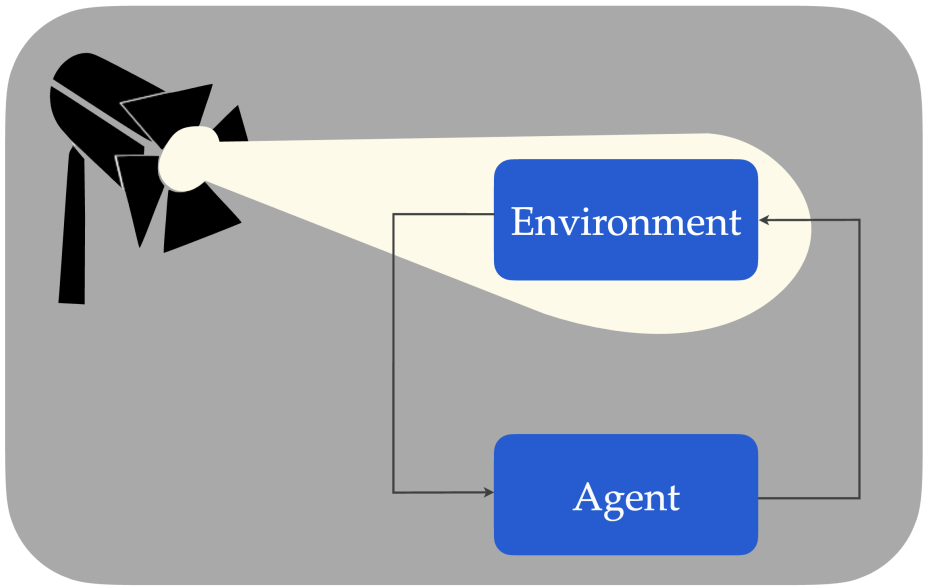
RL is different: Agent takes actions in an interactive environment, and receives scalar reward as feedback. This lends itself to interesting problems:

1. **Reward attribution:** Which action mattered?
2. **No ground truth:** Optimal policy unknown
3. **Explore vs. Exploit tradeoff:**
    1. **Exploration:** Learn how the world works
    2. **Exploitation:** Exploit best understanding to get reward
4. **(often) Online only:** No clear distinction between training and testing.
5. **Sparse reward:** Sparse feedback make things hard

What is RL really about...

Perhaps surprisingly, RL focuses a lot more on environments than on agents.

- Environments are fully described (e.g. as Markov Decision Processes).
- Agent is reflected by a policy (how to act).
- The agent is not part of the environment, being a disembodied locus of decision-making.



Markov decision processes (MDPs)

The **environment** is described by an MDP, which is tuple $(\mathcal{S}, T, \mathcal{A}, R, \gamma)$:

- A set of environmental states $\mathcal{S}$ and a set of actions $\mathcal{A}$
- *Distribution describing dynamics* $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$ of the form $T(s'|s, a) = \Pr(s_{t+1} = s'|s_t = s, a_t = a)$.
- *Reward function* $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$.
- *Discount factor* $\gamma \in [0, 1)$.

Markov decision processes (MDPs)

The **environment** is described by an MDP, which is tuple $(\mathcal{S}, T, \mathcal{A}, R, \gamma)$:

- A set of environmental states $\mathcal{S}$ and a set of actions $\mathcal{A}$
- *Distribution describing dynamics* $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$ of the form $T(s'|s, a) = \Pr(s_{t+1} = s'|s_t = s, a_t = a)$.
- *Reward function* $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$.
- *Discount factor* $\gamma \in [0, 1)$.

The **agent** is described by a *policy* $\pi : \mathcal{S} \to \Delta\mathcal{A}$. $\pi(a|s) = \Pr(a_t = a|s_t = s)$.

Markov decision processes (MDPs)

The **environment** is described by an MDP, which is tuple $(\mathcal{S}, T, \mathcal{A}, R, \gamma)$:

- A set of environmental states $\mathcal{S}$ and a set of actions $\mathcal{A}$
- *Distribution describing dynamics* $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$ of the form $T(s'|s, a) = \Pr(s_{t+1} = s'|s_t = s, a_t = a)$.
- *Reward function* $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$.
- *Discount factor* $\gamma \in [0, 1)$.

The **agent** is described by a *policy* $\pi : \mathcal{S} \to \Delta\mathcal{A}$. $\pi(a|s) = \Pr(a_t = a|s_t = s)$.

At each timestep t, the agent-environment loop works as follows:

- Agent samples $a_t \sim \pi(\cdot|s_t)$ and gives it to the environment.
- Environment samples $s_{t+1} \sim T(\cdot \mid s_t, a_t)$ and $R_{t+1} = R(s_t, a_t, s_{t+1})$, and gives (s_{t+1}, r_{t+1}) to agent.

Markov decision processes (MDPs)

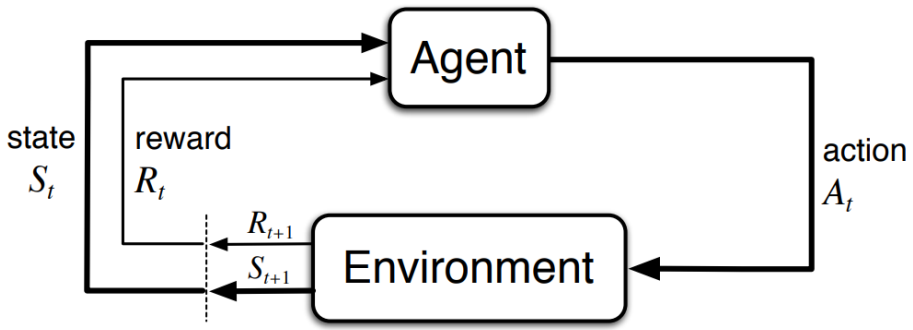
The **environment** is described by an MDP, which is tuple $(\mathcal{S}, T, \mathcal{A}, R, \gamma)$:

- A set of environmental states $\mathcal{S}$ and a set of actions $\mathcal{A}$
- *Distribution describing dynamics* $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$ of the form $T(s'|s, a) = \Pr(s_{t+1} = s'|s_t = s, a_t = a)$.
- *Reward function* $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$.
- *Discount factor* $\gamma \in [0, 1)$.

The **agent** is described by a *policy* $\pi : \mathcal{S} \to \Delta\mathcal{A}$. $\pi(a|s) = \Pr(a_t = a|s_t = s)$.

At each timestep t, the agent-environment loop works as follows:

- Agent samples $a_t \sim \pi(\cdot|s_t)$ and gives it to the environment.
- Environment samples $s_{t+1} \sim T(\cdot \mid s_t, a_t)$ and $R_{t+1} = R(s_t, a_t, s_{t+1})$, and gives (s_{t+1}, r_{t+1}) to agent.



Goal and assumptions

At timestep t, the *return* G_t is the sum of all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Goal and assumptions

At timestep t, the *return* G_t is the sum of all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Goal of the agent**: Maximise the expected return.

MDPs have a number of key 'nice' properties:

1. **Stationary:** The distribution T does not change over time.

Goal and assumptions

At timestep t, the *return* G_t is the sum of all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Goal of the agent**: Maximise the expected return.

MDPs have a number of key 'nice' properties:

1. **Stationary:** The distribution T does not change over time.
2. **Fully Observable:** The state is a full description of the world

Goal and assumptions

At timestep t, the *return* G_t is the sum of all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Goal of the agent**: Maximise the expected return.

MDPs have a number of key 'nice' properties:

1. **Stationary:** The distribution T does not change over time.
2. **Fully Observable:** The state is a full description of the world
3. **Markovian:** The environment dynamics depend only on the current state s_t and action a_t.

Goal and assumptions

At timestep t, the *return* G_t is the sum of all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Goal of the agent**: Maximise the expected return.

MDPs have a number of key 'nice' properties:

1. **Stationary:** The distribution T does not change over time.
2. **Fully Observable:** The state is a full description of the world
3. **Markovian:** The environment dynamics depend only on the current state s_t and action a_t.
4. **Reward Hypothesis:**

   "That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward)." — Richard Sutton

Value Function

- Want to define the "goodness" (*value*) of a state, so the agent can take actions to move towards "good" states, and away from "bad" states.

Value Function

- Want to define the "goodness" (*value*) of a state, so the agent can take actions to move towards "good" states, and away from "bad" states.
- The value of a state depends also on the current policy π chosen by the agent.

Value Function

- Want to define the "goodness" (*value*) of a state, so the agent can take actions to move towards "good" states, and away from "bad" states.
- The value of a state depends also on the current policy π chosen by the agent.

Value Function

$$V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

(Expectation is also with respect to the transitional distribution T.)

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \end{aligned}$$

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \end{aligned}$$

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s'|s,a) R(s,a,s') \\ &\quad + \gamma \sum_a \pi(a|s) \sum_{s'} T(s'|s,a) \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] \end{aligned}$$

Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s'|s,a) R(s,a,s') \\ &\quad + \gamma \sum_a \pi(a|s) \sum_{s'} T(s'|s,a) \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s'|s,a) \left(R(s,a,s') + \gamma V_\pi(s')\right) \end{aligned}$$

Policy Evaluation

This gives the **Bellman equation**

Bellman Equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s,a) \left(R(s,a,s') + \gamma V_\pi(s') \right)$$

Policy Evaluation

This gives the **Bellman equation**

Bellman Equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s,a) \left(R(s,a,s') + \gamma V_\pi(s') \right)$$

- Equation is linear in $V_\pi(\cdot)$, giving a set of **linear** simultaneous equations.

Policy Evaluation

This gives the **Bellman equation**

Bellman Equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s,a) \left(R(s,a,s') + \gamma V_\pi(s') \right)$$

- Equation is linear in $V_\pi(\cdot)$, giving a set of **linear** simultaneous equations.
- Given policy π, can now easily solve for $V_\pi(s_1), V_\pi(s_2), \ldots$

Policy Evaluation

This gives the **Bellman equation**

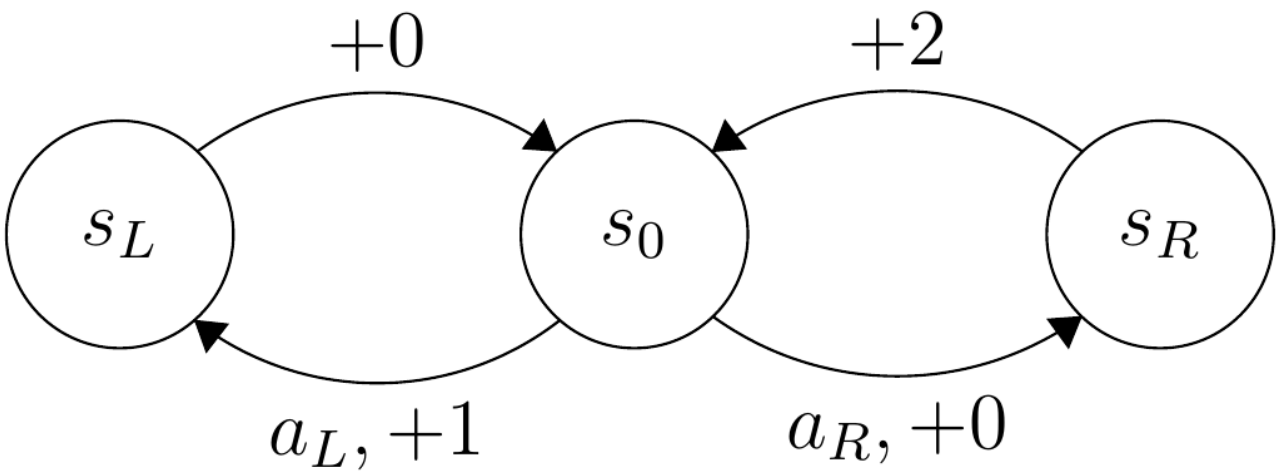
Bellman Equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s,a) \left(R(s,a,s') + \gamma V_\pi(s') \right)$$

- Equation is linear in $V_\pi(\cdot)$, giving a set of **linear** simultaneous equations.
- Given policy π, can now easily solve for $V_\pi(s_1), V_\pi(s_2), \ldots$
- Computing V_π from π is called **policy evaluation**, denote $\pi \xrightarrow{E} V_\pi$.

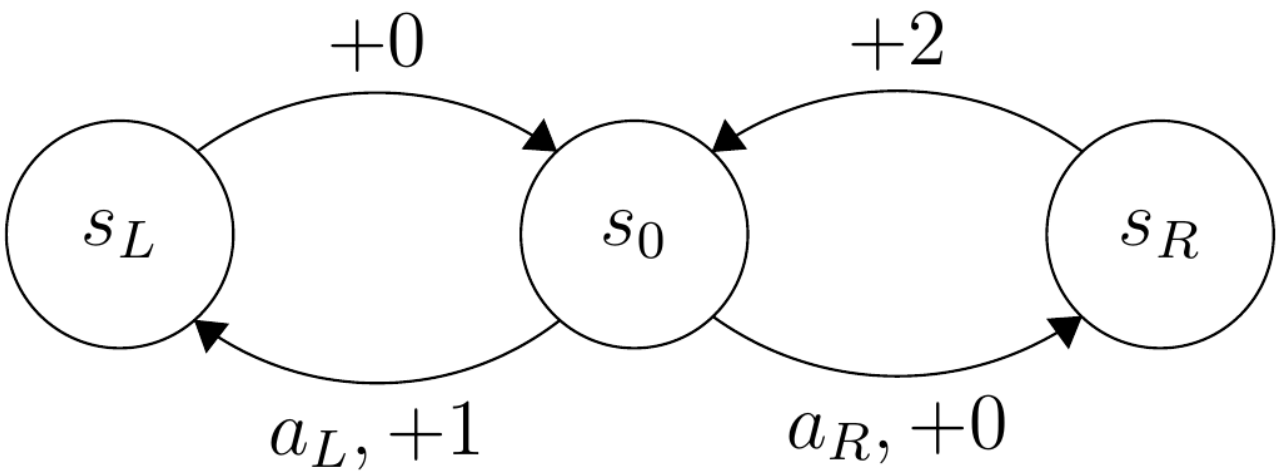
Example Environment

- States $\mathcal{S} = s_0, s_L, s_R$, actions $\mathcal{A} = \{a_L, a_R\}$, possible rewards $\mathcal{R} = \{0, 1, 2\}$.
- Each transition indicates if an action is taken, the reward returned and which state to transition to
- What is the best action from state s_0?



Example Environment

- States $\mathcal{S} = s_0, s_L, s_R$, actions $\mathcal{A} = \{a_L, a_R\}$, possible rewards $\mathcal{R} = \{0, 1, 2\}$.
- Each transition indicates if an action is taken, the reward returned and which state to transition to
- What is the best action from state s_0?



Exercise: Depends on the choice of γ! Verify that

$$V_{\pi_L}(s_0) = \frac{1}{1-\gamma^2} \quad V_{\pi_R}(s_0) = \frac{2\gamma}{1-\gamma^2}$$

so breakeven is at $\gamma = \frac{1}{2}$.

Optimal Bellman

- Policy π_1 is **better** than π_2 ($\pi_1 \geq \pi_2$) if $V_{\pi_1}(s) \geq V_{\pi_2}(s), \forall s \in \mathcal{S}$. A policy is **optimal** if it is better than all other policies.

Optimal Bellman

- Policy π_1 is **better** than π_2 ($\pi_1 \geq \pi_2$) if $V_{\pi_1}(s) \geq V_{\pi_2}(s), \forall s \in \mathcal{S}$. A policy is **optimal** if it is better than all other policies.
- **Theorem:** An optimal policy π^* always exists (may not be unique). Define optimal value function as

$$V_*(s) := \max_{\pi} V_\pi(s) \equiv V_{\pi^*}(s)$$

Optimal Bellman

- Policy π_1 is **better** than π_2 ($\pi_1 \geq \pi_2$) if $V_{\pi_1}(s) \geq V_{\pi_2}(s), \forall s \in \mathcal{S}$. A policy is **optimal** if it is better than all other policies.
- **Theorem:** An optimal policy π^* always exists (may not be unique). Define optimal value function as

$$V_*(s) := \max_\pi V_\pi(s) \equiv V_{\pi^*}(s)$$

Optimal Bellman Equation

$$V_*(s) = \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right)$$

Gives a set of **non-linear** simultaneous equations with variables $V_*(s_1), V_*(s_2), \ldots$

Policy Improvement

- Obviously we have that

$$V_*(s) = \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right) \tag{1}$$

$$\geq \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right) \tag{2}$$

$$\geq \sum_{s'} T(s'|s,\pi(s))\left(R(s,\pi(s),s') + \gamma V_\pi(s')\right) \tag{3}$$

$$= V_\pi(s) \tag{4}$$

Policy Improvement

- Obviously we have that

$$
\begin{aligned}
V_*(s) =& \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right) && (1)\\
\geq& \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right) && (2)\\
\geq& \sum_{s'} T(s'|s,\pi(s))\left(R(s,\pi(s),s') + \gamma V_\pi(s')\right) && (3)\\
=& V_\pi(s) && (4)
\end{aligned}
$$

- Given the value of a policy V_π, can feed it through the optimal Bellman equation (2) to get a better policy π'. Denote $V_\pi \xrightarrow{I} \pi'$.

Policy Improvement

- Obviously we have that

$$V_*(s) = \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right) \quad (1)$$

$$\geq \max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right) \quad (2)$$

$$\geq \sum_{s'} T(s'|s,\pi(s))\left(R(s,\pi(s),s') + \gamma V_\pi(s')\right) \quad (3)$$

$$= V_\pi(s) \quad (4)$$

- Given the value of a policy V_π, can feed it through the optimal Bellman equation (2) to get a better policy π'. Denote $V_\pi \xrightarrow{I} \pi'$.

Policy Improvement

$$\pi'(s) := \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right)$$

Exercise: Prove $\pi' \geq \pi$.

Policy Iteration

Policy Improvement (I)

$$\pi'(s) \leftarrow \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right)$$

Policy Evaluation (E)

Solve

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right)$$

for $V_\pi(s_1), V_\pi(s_2), \ldots$.

Policy Iteration

Policy Improvement (I)

$$\pi'(s) \leftarrow \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right)$$

Policy Evaluation (E)

Solve

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_\pi(s')\right)$$

for $V_\pi(s_1), V_\pi(s_2), \ldots$.

- Start with arbitrary policy π_0.
- Note that π_* is fixed point of policy improvement.
- Alternate until policy is stable

$$\pi_0 \xrightarrow{E} V_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} V_{\pi_2} \xrightarrow{I} \ldots \xrightarrow{I} \pi_* \xrightarrow{E} V_{\pi^*} \xrightarrow{I} \pi_*$$

Model-free agents: Temporal difference learning

Goal: Do policy evaluation without knowing the environment's dynamics.

- **Motivation:** Consider once again the value function:

$$V_\pi(s) = \mathop{\mathbb{E}}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s,a,s') + \gamma V_\pi(s')]$$

Model-free agents: Temporal difference learning

Goal: Do policy evaluation without knowing the environment's dynamics.

- **Motivation:** Consider once again the value function:

$$V_\pi(s) = \mathop{\mathbb{E}}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s,a,s') + \gamma V_\pi(s')]$$

This means that $V_\pi(s_t) \approx r_{t+1} + \gamma V_\pi(s_{t+1})$.

Model-free agents: Temporal difference learning

Goal: Do policy evaluation without knowing the environment's dynamics.

- **Motivation:** Consider once again the value function:

$$V_\pi(s) = \mathop{\mathbb{E}}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s,a,s') + \gamma V_\pi(s')]$$

  This means that $V_\pi(s_t) \approx r_{t+1} + \gamma V_\pi(s_{t+1})$.

We define the **TD-Error** as the difference

$$\delta_t := \Big[r_{t+1} + \gamma V_\pi(s_{t+1}) \Big] - V_\pi(s_t).$$

Model-free agents: Temporal difference learning

Goal: Do policy evaluation without knowing the environment's dynamics.

- **Motivation:** Consider once again the value function:

$$V_\pi(s) = \mathop{\mathbb{E}}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s,a,s') + \gamma V_\pi(s')]$$

  This means that $V_\pi(s_t) \approx r_{t+1} + \gamma V_\pi(s_{t+1})$.

We define the **TD-Error** as the difference

$$\delta_t := \Big[r_{t+1} + \gamma V_\pi(s_{t+1})\Big] - V_\pi(s_t).$$

This suggests the following update rule to improve on our estimation:

$$\hat{V}_\pi(s_t) \leftarrow \hat{V}_\pi(s_t) + \alpha\delta_t = \hat{V}_\pi(s_t) + \alpha\left(r_{t+1} + \gamma\hat{V}_\pi(s_{t+1}) - \hat{V}_\pi(s_t)\right),$$

where $\alpha \in (0, 1]$ is the **learning rate**.

Theorem: If $\alpha \to 0$ in a nice way, $\hat{V}_\pi$ is guaranteed to converge to V_π.

Q-Value

- **Q-value** is the expected return from state s, taking action a, and thereafter following policy π.

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

Q-Value

- **Q-value** is the expected return from state s, taking action a, and thereafter following policy π.

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

Contrast with the value function

$$V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

Q-Value

- **Q-value** is the expected return from state s, taking action a, and thereafter following policy π.

$$Q_\pi(s,a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

Contrast with the value function

$$V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

Q-value Bellman

$$Q_\pi(s,a) = \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma Q_\pi(s',a')\right)$$

where $a' = \pi(s')$

Optimal Q-value Bellman

$$Q_*(s,a) = \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma \max_{a'} Q_*(s',a')\right)$$

Motivation for Q-learning

- So far, we have been learning a policy π, and using π to compute V_π.

Motivation for Q-learning

- So far, we have been learning a policy π, and using π to compute V_π.
- Even if we were given V_* directly, can't recover π_* without white-box access to T and R (environment).

$$\pi_*(s) = \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right)$$

Motivation for Q-learning

- So far, we have been learning a policy π, and using π to compute V_π.
- Even if we were given V_* directly, can't recover π_* without white-box access to T and R (environment).

$$\pi_*(s) = \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right)$$

- However, given Q_*, we can directly recover π_*

$$\pi_*(s) = \arg\max_a Q_*(s,a)$$

Motivation for Q-learning

- So far, we have been learning a policy π, and using π to compute V_π.
- Even if we were given V_* directly, can't recover π_* without white-box access to T and R (environment).

$$\pi_*(s) = \arg\max_a \sum_{s'} T(s'|s,a)\left(R(s,a,s') + \gamma V_*(s')\right)$$

- However, given Q_*, we can directly recover π_*

$$\pi_*(s) = \arg\max_a Q_*(s,a)$$

- **Idea:** Learn Q_* instead, argmax to recover policy π_*

Summary of key ideas

- **Environment:** MDP (Markovian environment affected by actions, which provides rewards and a discount factor)
- **Agent:** policy π (deterministic or stochastic)
- **Value function $V_\pi(s)$:** the expected return from state s under policy π, which is the thing the agent wants to maximise.
- **Q-value function $Q_\pi(s, a)$:** the expected return from state s taking action a and thereafter following policy π.
- **Bellman equations:** characterise the value function and Q-value function.
- **Model-based agents:** use the environment's dynamics to build a good policy.
- **Model-free agents:** build good policies without knowledge of the environment's dynamics.
- **Deep RL:** estimating value functions using neural networks.

Types of weak agents

Types of weak agents:

- **Memoryless agents:** take actions based on the present state of the environment.
- **Non-plastic agents:** take actions based on a hardcoded state-to-action lookup table.

Types of weak agents

Types of weak agents:

- **Memoryless agents:** take actions based on the present state of the environment.
- **Non-plastic agents:** take actions based on a hardcoded state-to-action lookup table.

Memoryless agents are optimal in fully observable (i.e. Markovian) environments.

Non-plastic agents can be optimal but cannot deal with change (non-stationarity).

Most powerful agents are memory-full and plastic...

Model-free vs model-based agents

Type	**Estimate dynamics**	**World model**	**Data efficiency**
Model-free	No	Minimal	Experience-hungry
Model-based	Yes	Complex	Experience-efficient

- Model-free agents can learn any environment, but need to try more things.

Model-free vs model-based agents

Type	Estimate dynamics	World model	Data efficiency
Model-free	No	Minimal	Experience-hungry
Model-based	Yes	Complex	Experience-efficient

- Model-free agents can learn any environment, but need to try more things.
- Model-based agents can use world model to 'imagine' and plan without taking actions.

Model-free vs model-based agents

Type	Estimate dynamics	World model	Data efficiency
Model-free	No	Minimal	Experience-hungry
Model-based	Yes	Complex	Experience-efficient

- Model-free agents can learn any environment, but need to try more things.
- Model-based agents can use world model to 'imagine' and plan without taking actions.
- Traps (absorbing states) are bad for model-free agents.

Types of highly-capable agents

1. **Meta-cognitive agents:** with a sense of confidence on their capabilities and limitations.

Types of highly-capable agents

1. **Meta-cognitive agents:** with a sense of confidence on their capabilities and limitations.
2. **Self-aware agents:** with a world model that has a model of themselves.

Types of highly-capable agents

1. **Meta-cognitive agents:** with a sense of confidence on their capabilities and limitations.
2. **Self-aware agents:** with a world model that has a model of themselves.
3. **Self-modifying agents:** agents that can change their capabilities (RL doesn't work here).

Types of highly-capable agents

1. **Meta-cognitive agents:** with a sense of confidence on their capabilities and limitations.
2. **Self-aware agents:** with a world model that has a model of themselves.
3. **Self-modifying agents:** agents that can change their capabilities (RL doesn't work here).
4. **Self-replicating agents:** can make multiple copies that can coordinate with each other (RL doesn't work here).

Types of highly-capable agents

1. **Meta-cognitive agents:** with a sense of confidence on their capabilities and limitations.
2. **Self-aware agents:** with a world model that has a model of themselves.
3. **Self-modifying agents:** agents that can change their capabilities (RL doesn't work here).
4. **Self-replicating agents:** can make multiple copies that can coordinate with each other (RL doesn't work here).
5. **Sentient agents:** agents that care — can feel pleasure and pain.

Most of these cannot be properly studied with vanilla RL...

We need other frameworks — stay tuned for the next sessions of Iliad Intensive!